תכנות מונחה עצמים – תרגיל 3

תאריך ההגשה: 14.12.25 בשעה 23:55

תרגיל זה מומלץ להגיש בזוגות.

לדרישות התרגיל לחצו פה.

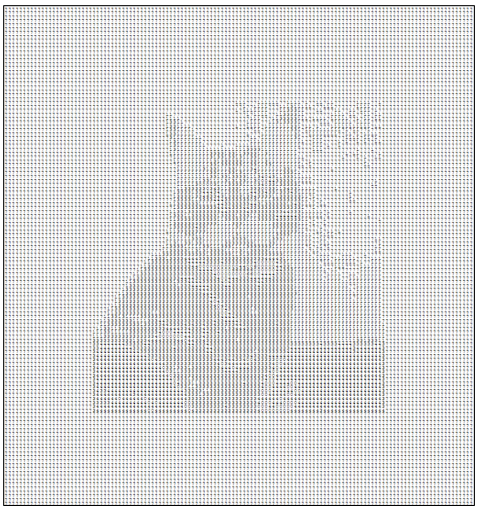
0. הקדמה

בתרגיל נבנה תוכנה הממירה קבצי תמונה לאמנות ASCII. התוכנה תקבל כקלט כתובת לקובץ תמונה, אוסף תווי ASCII בהם נרצה להשתמש בהמרה, וכן את הרזולוציה שבה נרצה להמיר את התמונה, ותמיר אותה לתמונת ASCII. הקלט יודפס כתמונה לקונסול או יישמר כקובץ html. נוסף על כך, נבנה ממשק משתמש.
דוגמה להמרה של תמונה בשני מקרים, 1. המרה עם תווי הספרות 0-9 הקיימים ברזולוציה של 128 תווים בשורה 2. המרה עם כל התווים בטבלת ASCII ברזולוציה 256.

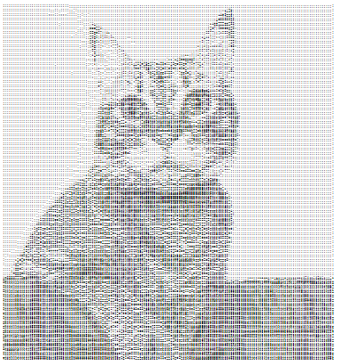
תמונת המקור



תמונת היעד מקרה 1



תמונת היעד מקרה 2

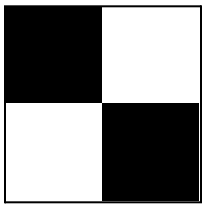


התרגיל מורכב משני חלקים: בחלק הראשון נפתח את אלגוריתם אומנות האסקי, ובחלק השני נפתח ממשק משתמש שבו יבחרו הפרמטרים להמרה ע"י המשתמש. יש להגיש את שניהם יחד במועד ההגשה.
שימו ❤️: קבצי העזר לשימוש בתרגיל זה מופיעים ב-moodle.
לפני שניגש לתרגיל עצמו, נציג בחלקים הבאים של ההקדמה כמה נושאי רקע שחשוב להכיר לפני תחילת העבודה על התרגיל.

0.1. ייצוג תמונות במחשב

כדי לייצג תמונות במחשב, אנחנו נחזיק מערך דו מימדי אשר כל תא במערך מיצג פיקסל בתמונה והוא מכיל מידע על הצבע של הפיקסל. הצבעים מורכבים משלושה צבעי יסוד – ירוק (G), אדום (R) וכחול (B) – ונקבעים על פי עוצמת ההארה של שלושת הצבעים הללו. כל עוצמת הארה תהיה בסולם של 0-255. לפיכך, נסמן כל צבע כשלשה של מספרים (R,G,B) כאשר כל מספר מסמן את עוצמת ההארה של צבע היסוד שלו. בשפת java, כל תא במערך שלנו יחזיק אובייקט של המחלקה Color, ולכל מופע של Color יהיו שדות של שלושת הצבעים (לא תצטרכו לממש מחלקה זו).

לדוגמא, השלשה (0,0,0) מייצגת צבע שחור, ואילו השלשה (255,255,255) מייצגת צבע לבן.
כעת, אם נביט על התמונה הזו:



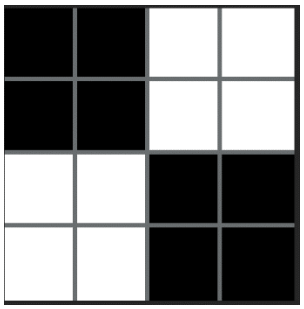
נוכל להציג אותה כמערך בגודל 2*2:
שורה ראשונה:

```
img[0][0] = Color(0,0,0), img[0][1] = Color(255,255,255)
```

שורה שנייה:

```
img[1][0] = Color(255,255,255), img[1][1] = Color(0,0,0)
```

באופן דומה, ניתן לייצג תמונה זו כמערך של 4*4 פיקסלים:



מספר הפיקסלים שמייצגים את התמונה מגדירים את הרזולוציה שלה. ככל שהתמונה מיוצגת על ידי יותר פיקסלים, נגיד שהרזולוציה שלה גבוהה יותר, והיא תראה "חדה" יותר.

0.2. תווים

בג'אווה (וגם בשפות נוספות), char הוא משתנה פרימיטיבי המחזיק ערך מספרי כלשהו המייצג תו. תוכלו לראות את ההמרה בין התווים למספרים באמצעות חיפוש פשוט בגוגל של טבלת ASCII (לדוגמה, התו a מיוצג על ידי המספר 65). ככזה, ג'אווה מאפשרת לנו לבצע autocasting בין char ל-int כך שנוכל לבצע המרה בכיוון השני בצורה מפורשת. תוכלו להשתמש בתכונה הזו בעת כתיבת התרגיל.

0.3. ASCII ART

סוג אומנות שנוצרת מאותיות ascii, על ידי המרה של קבוצת פיקסלים לאות ascii שדומה לה בבהירות (ראה דוגמאות בתחילת ההקדמה). מס׳ הפיקסלים שמומרים בתו הוא פרמטר של התהליך; ככל שממירים מספר נמוך יותר של פיקסלים לאות, ככה נקבל תמונה שדומה יותר לתמונה המקורית. שימו לב שאין הכרח להשתמש בכל תווי ה-ascii, אלא ניתן להגדיר תת קבוצה של תווים שנשתמש רק בהם. בנוסף, שימו לב כי בכל גופן (-), לתו יש נראות מעט שונה, ולכן התמונה תיראה קצת שונה בגופנים שונים. הסבר ופירוט נוסף ניתן לראות בקישור לויקיפדיה.

1.חלק א׳: אלגוריתם אומנות ה-ASCII

בחלק זה של התרגיל נפתח את האלגוריתם בשלבים:

1.1. ראשית, נעבור במבט על על שלבי האלגוריתם לפני שנכנס לפרטים.
1.2. לאחר מכן נציג את עיצוב העל של התרגיל שהוא החלוקה לחבילות לפי תחומי האחריות של הקוד. את מבנה החלוקה של התרגיל לחבילות שונות אנחנו מספקים לכם, ובתוך שלד העיצוב הזה תידרשו לעשות בחירות עיצוביות מקומיות.
1.3. לאחר החלוקה לחבילות נציג את הקוד המסופק לכם שאותו אין צורך לממש.
1.4. לבסוף נציג בצורה מסודרת את דרישות התרגיל לצורך מימוש האלגוריתם.

1.1. שלבי האלגוריתם

1.1.1. שלב קדם - קביעת בהירות תווי ה-ASCII:

עבור כל תו בסט התווים שנבחר לצורך ההמרה, נחשב את הבהירות שלו (באמצעות נוסחה שתפורט בהמשך). את הבהירויות של התווים נשמור במבנה נתונים לבחירתכם. את השלב הזה נבצע פעם אחת עבור סט תווים נתון, ובעת הצורך נפנה למבנה הנתונים. הדגש שלנו בתרגיל הזה הוא גם על יעילות בזמני ריצה וזיכרון, לכן תצטרכו לחשוב על מבנה או מבני הנתונים המתאימים ביותר.

1.1.2. שלב א' - ריפוד התמונה:

בשלב הראשון נרפד את התמונה בפיקסלים לבנים (במידת הצורך) כדי שאורכה ורוחבה יהיו חזקות של 2 ונוכל לחלק אותה לריבועים.

1.1.3. שלב ב׳ - המרת התמונה לתתי תמונות כתלות ברזולוציה:

כל תמונה אפשר לייצג ברזולוציות שונות – נוכל להמיר כל פיקסל בודד לתו ASCII, אך ייתכן שנרצה להמיר ריבוע פיקסלים בגודל כלשהו לתו ASCII יחיד. הרזולוציה של ההמרה שלנו תוגדר ע"י מספר התווים שאנחנו רוצים להכניס בשורה בתמונת היעד, ובהתאם לכך, יוגדר גם מספר הריבועים שאליו תחולק התמונה כך שכל תת תמונה תוחלף בתו ASCII בודד. ככל שהתמונה המקורית תחולק ליותר תתי-תמונות כך הרזולוציה תהיה גבוהה יותר ותמונת ה-ASCII תראה דומה יותר לתמונה המקורית.

בהנתן רזולוציה שנבחרה ע"י המשתמש, נחלק את התמונה המקורית לתתי תמונות של ריבועים בגודל המתאים (כך שתתי התמונות מרכיבות יחד את התמונה המקורית).

1.1.4. שלב ג' - עבור כל תת תמונה המרה לתו בבהירות מתאימה:

עבור כל ריבוע (תת תמונה), נחשב את עוצמת הבהירות שלה (בנוסחה אותה נפרט בהמשך). לאחר מכן, נחליף את תת התמונה בתו ASCII עם הבהירות הדומה ביותר.

1.2. חלוקה לחבילות

חלוקה לחבילות היא שלב ראשון בתכנון העיצוב של התוכנה. אנחנו נחלק את התוכנה שלנו ל-4 חבילות (מודולים) לפני תחומי האחריות הבאים:

1.2.1. החבילה image:

תכיל את כל הלוגיקה של עיבוד התמונה. כפי שתואר בחלקים הקודמים, אנו רוצים לרפד את התמונה וכן לחלק את התמונה לתתי תמונות ולחשב ערך בהירות של תתי תמונות.

1.2.2. החבילה ascii_art:

החבילה שתהיה אחראית על הרצת האלגוריתם.

1.2.3. החבילה image_char_matching:

החבילה שתהיה אחראית על התאמה של תת תמונה לתו בבהירות הכי קרובה. חבילה זו תשתמש בחבילה image בכל הקשור לעבודה עם התמונות.

1.2.4. החבילה ascii_output:

חבילה שמטפלת בהצגת התמונה. חבילה זו מסופקת לכם במלואה.

1.3. הקוד המסופק

1.3.1. המחלקה `KeyboardInput` בחבילה ascii_art:

המחלקה מכילה את המתודה readLine לצורך קבלת קלט מהמשתמש. נשתמש בה בחלק השני של התרגיל. (בדומה לתרגיל 1)

1.3.2. המחלקה `CharConverter` בחבילה image_char_matching:

המחלקה מכילה מתודה סטטית פומבית אחת שממירה תו ASCII למערך בוליאני המייצג את תמונת התו בשחור לבן :

```
public static boolean[][] convertToBoolArray(char c)
```

בהינתן תו מסוים, המתודה תחזיר מערך בוליאני דו מימדי (בגודל 16X16). מערך זה מייצג את התמונה בשחור-לבן של התו בגופן `Courier New` (נעבוד רק עם הגופן הזה במהלך כל התרגיל) כאשר הערך `true` מייצג פיקסל לבן ו-`false` מייצג פיקסל שחור. בפונקציה זו נעזר כאשר נרצה לחשב את ערך הבהירות של התווים (שיהיה בעצם מספר הפיקסלים הלבנים מתוך כל הפיקסלים).

שימו לב: סטודנטים המשתמשים במחשבי Mac (אולי גם במחשבים נוספים) עלולים לראות הבדל ברינדור של תווים ולכן הבהירות יוצאת שונה. אנא בדקו את הקוד שלכם על מחשבי בית הספר.

1.3.3. המחלקה `Image` בחבילה image:

מחלקה המייצגת תמונה במחשב ע"י מערך דו מימדי של אובייקטים מסוג `Color` (כמו שמתואר בהקדמה). למחלקה יש שני בנאים:

1. בנאי המקבל שם של קובץ תמונה וטוען את הפיקסלים של התמונה לתוך המערך. הבנאי זה יכול לזרוק `IOException` אם יש בעיה עם קובץ התמונה. שימו לב שמי שקורא לבנאי צריך לדאוג שיש מי שתופס את החריגה ומטפל בה אם היא נזרקת.
2. בנאי המקבל מערך דו מימדי של `Color`, וכן את אורך ורוחב התמונה המיוצגת במערך.

למחלקה ארבע מתודות פומביות:

1. `getWidth` המחזירה את רוחב התמונה בפיקסלים
2. `getHeight` המחזירה את אורך התמונה בפיקסלים
3. `getPixel(int x, int y)` המחזיקה את הפיקסל `(x,y)` במערך התמונה.
4. `saveImage(String fileName)` השומרת את התמונה בקובץ בשם `fileName` עם הסיומת jepg.

1.3.4. החבילה ascii_output:

החבילה ascii_output מסופקת לכם במלואה כדי שתוכלו לייצא בקלות את הפלט של האלגוריתם בצורה הנוחה לכם. החבילה מכילה את הקבצים הבאים:

1. הממשק `AsciiOutput` המגדיר את המתודה `out` שבהינתן מערך תווים דו מימדי מייצא אותו לפלט בהתאם למימוש.
2. המחלקה `ConsoleAsciiOutput` שיודעת לייצא מערך תווים דו מימדי לקונסול (חלון ההרצה)
3. המחלה `HtmlAsciiOutput` שיודעת לייצא מערך תווים דו מימדי לקובץ בשם out.html כך שיהיה ניתן להציג את תמונת הascii בדפדפן ויהיה אפשר לעשות לה zoom-in/zoom-out.

שימו לב! אין לבצע שינויים בקוד של חבילה זו!

1.4. דרישות

למטה מתוארות דרישות התרגיל. בחלק מהדרישות לא ניתן לכם API ספציפי לממש, אלא רק הסבר על מה הקוד אמור לעשות. במקרים אלו, חישבו ראשית מה הקלט והפלט הרצויים, ובמידה וזה לא צויין – באיזו מחלקה/מודול (חדשים או קיימים) כדאי לממש את הדרישה. קחו בחשבון היכן משתלב הקוד הזה: מי קורא לו ולמי הוא קורא. לאחר מכן ממשו את הדרישות.
שימו ❤️:

- על המימוש להיות הגיוני ומתאים, ולעמוד בכללים והעקרונות הנלמדים בקורס.
- יש לקרוא את התרגיל עד הסוף כולל חלק ב' שבו מוסבר על ממשק המשתמש לפני מימוש הדרישות. חשוב מאוד לקבל את התמונה השלמה של התוצר הסופי כדי לבחור בחירות עיצוביות נכונות.

- תכננו את הקוד שלכם בעזרת דיאגרמת UML, אתם תדרשו להגיש את הדיאגרמה המתאימה לקוד שלכם בהגשת התרגיל ולהסביר את העיצוב שלכם ב-README.
- בשלב הראשון של התרגיל, נממש את האלגוריתם עם ערכי ברירת מחדל, ובשלב השני של התרגיל נוסיף את הממשק שבו המתשמש יוכל לשנות את הפרמטרים.

1.4.1. שלב קדם: קביעת בהירות תווי ה-ASCII:

1.4.1.1. הוסיפו לחבילה image_char_matching את המחלקה `SubImgCharMatcher` שתהיה אחראית להתאים תו ASCII לתת תמונה עם בהירות נתונה. מחלקה זו תשמש את אלגוריתם אומנות ה-ASCII שלנו בהמשך כדי להחליף תתי תמונות בתווים.

לפני שנסביר איך לחשב את ערך הבהירות של תו ASCII, נציג ה-API של המחלקה:

1. `public SubImgCharMatcher(char[] charset)`
   בנאי המקבל כפרמטר מערך של תווים שיהוו את סט התווים לשימוש האלגוריתם.

2. `public char getCharByImageBrightness(double brightness)`
   בהינתן ערך בהירות `brightness` (של תת תמונה), המתודה תחזיר את התו (מתוך סט התווים) עם הבהירות הכי קרובה בערך מוחלט לבהירות הנתונה.
   שימו לב! אם יש מספר תווים מתוך הסט בעלי בהירות זהה, יוחזר התו עם ערך הASCII הנמוך מביניהם.

3. `public void addChar(char c)`
   מתודה שמוסיפה את התו `c` לסט התווים.

4. `public void removeChar(char c)`
   מתודה שמסירה את התו `c` מסט התווים.

5. מתודות פומביות נוספות לפי הצורך. אם תוסיפו מתודות ל-API, תתבקשו להסביר את הסיבה לכך ב-README.

המימוש של המחלקה נתון לשיקול דעתכם. תוכלו כמובן להוסיף למחלקה שדות ומתודות פרטיים לפי הצורך, וכן להוסיף מחלקות נוספות לחבילה image_char_matching שישמשו את המחלקה `SubImgCharMatcher`. חשבו מראש באילו מבני נתונים תרצו לשמור את הנתונים והביאו בחשבון שיקולי יעילות וזיכרון. אתם תתבקשו להסביר ב-README באילו מבני נתונים השתמשתם ומדוע.

1.4.1.2. כעת נסביר איך לחשב את ערך הבהירות של תו ASCII:
עבור כל תו בודד:

1. ראשית נמיר את התו למערך דו מימדי בוליאני בגודל 16X16 בעזרת המתודה `CharConverter.convertToBoolArray` המסופקת (ר׳ לעיל). לאחר מכן נספור את ערכי התאים בהם הערך הוא `true` (אינדיקציה שזהו פיקסל בצבע לבן). שימו לב- בתרגיל הזה תמיד נרנדר את התווים לפי רזולוציה של 16 פיקסלים בלי תלות ברזולוציה של התמונה ושל תת התמונות.
2. ננרמל את המספר שקיבלנו בסך מספר הפיקסלים במערך על מנת לקבל מספר בין 0-1 המייצג את רמת הבהירות של התו.

לאחר שחישבנו את ערך הבהירות של כל התווים:

3. ננרמל את כל ערכי הבהירות של התווים שחישבנו ע״י מתיחה ליניארית של ערכי הקצוות בסט התווים הנתון. זאת על מנת למנוע מצב שבתמונה בעלת גוונים דומים נקבל את אותה האות לכל פיקסל בתמונה. המתיחה מחדדת לנו את הבדלי הבהירות בין האותיות השונות וכך אנחנו נמנעים ממצב כזה. את המתיחה נבצע לפי החישוב הבא:

$$newCharBrightness = \frac{charBrightness - minBrightness}{maxBrightness - minBrightness}$$

- *שימו לב שכדי לבצע את שלב הנרמול (3) יש צורך לחשב קודם כל את ערכי הבהירות של כל התווים בסט התווים כדי למצוא את הבהירות המינימלית ואת הבהירות המקסימלית מתוך כל התווים.*
- *הניחו כי לא תתבקשו לנרמל סט עם תו אחד בלבד (המכנה מתאפס) וכן ניתן להניח שיש לפחות שני תוים עם ערך בהירות שונה בסט.*

1.4.2. שלב א׳+ב׳: ריפוד התמונה וחלוקת התמונה לתתי תמונות:

עבור שלבים אלו לא נדרוש מכם API ספציפי, והעיצוב נתון לשיקולכם. עליכם להוסיף את הפונקציונליות המפורטת מטה ע"י הוספת מחלקה או מחלקות מתאימות לחבילה image.

1.4.2.1. ריפוד תמונה בפיקסלים לבנים כך שהיא תהיה באורך וברוחב של חזקה של 2 (זו שגדולה אך קרובה ביותר למקורי, התמונה לא צריכה להפוך לריבוע).
למשל: תמונה שגודלה 30x14 נגדיל את התמונה להיות 32x16 בעזרת ריבועים לבנים בקצוות.

- הריפוד צריך להיות סימטרי בין הצדדים השונים (אפשר להניח שהמימדים של הקלט תמיד זוגיים).
- אם אחד המימדים של התמונה הוא חזקה של 2, אין צורך לרפד אותו.

1.4.2.2. חלוקה של תמונה לתת תמונות לפי רזולוציה נתונה (כך שתתי התמונות מרכיבות יחד את התמונה המקורית והן לא חופפות). הקלט יהיה הרזולוציה הרצויה, כלומר מספר תתי התמונות בשורה, והפלט יהיה מבנה נתונים המכיל את כל תתי התמונות של התמונה (כל תת תמונה היא ריבוע של פיקסלים). בהמשך כל תת תמונה שכזו תומר לתו ASCII. המימוש הספציפי הוא כראות עיניכם. ניתן להניח שהרזולוציה הנתונה תקינה.

1.4.2.3. חישוב בהירות של תמונה/תת תמונה.

- ראשית יש להמיר כל פיקסל בתמונה לגוון אפור. כדי לעשות זאת, נמיר כל אובייקט מסוג `Color` למספר בין 0-255. לצורך ההמרה נשתמש בנוסחה הבאה:

```
greyPixel = color.getRed() * 0.2126 + color.getGreen() * 0.7152
                                    + color.getBlue() * 0.0722
```

- לאחר מכן יש לסכום את כל גווני האפור של הפיקסלים ולחלק בסך הפיקסלים בתת-התמונה ובציון ה-RGB המקסימאלי 255. (על מנת לנרמל את הגוון האפור להיות בין 0-1).

1.4.3. שלב ג': מימוש אלגוריתם ה-ASCII ART עצמו

הוסיפו את המחלקה `AsciiArtAlgorithm` לחבילה ascii_art. מחלקה זו תהיה אחראית על ריצה בודדת של האלגוריתם (עבור כל הרצה של האלגוריתם יש ליצור מופע חדש של המחלקה). הפרמטרים עבור ריצת האלגוריתם יתקבלו בבנאי של המחלקה. הריצה תהיה תלויה כמובן בתמונה שנבחרה, ברזולוציה שנבחרה (כמה תווים יהיו בשורה בתוצאה), וכן בסט התווים שנבחר. **הדרך שבה נתונים אלו יתורגמו לפרמטרים של המחלקה תלוי במימוש שלכם לחלקים הקודמים.**

מלבד הבנאי למחלקה תהיה מתודה פומבית אחת בלבד (אין לשנות את חתימת המתודה):

```
public char [][] run()
```

המתודה מחזירה מערך דו מימדי של תווים (ASCII) המייצגים תמונה (אותה תמונה שהתקבלה בבנאי). כל תת תמונה תוחלף בתו עם רמת הבהירות הקרובה ביותר בערך מוחלט כברירת מחדל.

1.5. הערה על יעילות האלגוריתם

נשים לב כי ניתן לייעל את הקוד כדי לחסוך בזמן ריצה של חישובים חוזרים בהרצות חוזרות של האלגוריתם בכמה היבטים:

1.5.1. נרצה לנסות ולחסוך בחישובים חוזרים של הבהירות של תתי תמונות. למעשה, אם אנחנו מתכוונים לקרוא שוב לאלגוריתם עבור אותה תמונה, אותה רזולוציה (כאשר רק מאגר התווים השתנה), נוכל לשמור את ערכי הבהירות שחושבו בריצה הקודמת ולהשתמש בהם בפעם הבאה שנרוץ על אותה תמונה עם אותה רזולוציה. (אין צורך לשמור תוצאות של יותר מריצה אחת אחורה)

1.5.2. נרצה לנסות ולחסוך בחישוב חוזר של הבהירות של תווי האסקי שנבחרו לסט התווים, שכן הבהירות של התווים (לפני הנרמול) לא משתנה בין ריצה לריצה.

1.5.3. נרצה לנסות ולחסוך בחישוב חוזר של הבהירות המנורמלת של התווים אם סט התווים לא השתנה בין ריצה לריצה. שימו לב שאם סט התווים כן השתנה, ייתכן והבהירות המנורמלת של התווים תשתנה אם ערכי המינימום/המקסימום של הבהירות בסט השתנו בעקבות הוספה או מחיקה של תווים מהסט.

מצופה מכם לייעל את הקוד לפי מנגנון שעושה שימוש במבני נתונים כדי לחסוך חישובים חוזרים. אתם תידרשו להסביר את הבחירה שלכם בקובץ ה-README בהתבסס על סיבוכיות זמן הריצה והזיכרון של המבנים בהם השתמשתם. בתרגיל הזה אתם רשאים להשתמש בספריית ה-collections או כל מבנה נתונים אחר שמסופק בספריות הסטנדרטיות של ג'אווה.

1.6. בדיקת שפיות

הריצו את הקוד שלכם עם הפרמטרים הבאים:

- התמונה תהיה "`board.jpeg`" המסופקת לכם בקבצים
- סט התווים יהיה 'o','m'
- הרזולוציה תהיה 2 תווים בשורה

ובדקו שהפלט הוא:

```
[[m, o], [o, m]]
```

בדיקות נוספות תוכלו לערוך מול הדוגמאות של תמונת החתול המובאות לכם בהקדמה עם הפרמטרים המתאימים. כמובן שבדיקות אלו לא מספיקות כדי לוודא שהקוד שלכם תקין, אבל הן מקום טוב להתחיל ממנו.

2. חלק ב׳ - ממשק משתמש

בחלק זה ניצור ממשק משתמש שבו המשתמש יוכל לבחור ולשחק עם הפרמטרים של אלגוריתם ה-ASCII art. התוכנית תקבל כקלט בשורת הריצה את המסלול לקובץ התמונה, ותתחיל עם ערכי ברירת המחדל שהזכרנו בחלק 1.4: סט התווים יכיל את תווי הספרות 0-9 בלבד, והרזולוציה תהיה 2, ואופן העיגול יהיו לפי ערך מוחלט. בנוסף, ברירת המחדל הפלט שלנו יודפס למסך.

הפקודות הבסיסיות שאנחנו רוצים לייצר לממשק שלנו הן:

1. `exit` - יציאה מהתוכנה.
2. `chars` -צפייה במאגר התווים הנוכחי.
3. `add` - הוספה תווים לסט התווים הנוכחי.
4. `remove` - הסרה של תווים מסט התווים הנוכחי.
5. `res` - שליטה ברזולוציה של התמונה שלנו – הגדלה והקטנה של הרזולוציה.
6. `output` - בחירת ה-Output – אם התוצר יודפס ל-console או לחילופין יוחזר כקובץ html
7. להפוך את הבהירות שמחפשים להיות המשלים שלו.
8. `asciiArt` - הרצת האלגוריתם על הפרמטרים הנוכחיים.

2.1. יצירת ממשק המשתמש

ראשית, נייצר מחלקה חדשה בשם Shell בחבילה ascii_art. המחלקה תבצע את כל פעולות ממשק המשתמש. ה-API של המחלקה תכיל את המתודות:

- main
- בנאי
- ובנוסף מתודה פומבית אחת בלבד:

```
public void run(String imageName)
```

אשר תפקידה לתרגם את הפקודות המתקבלות מהמשתמש ולבצע אותן.

יש להריץ את פקודת התוכנית עם המסלול לתמונה הרצויה כארגומנט. אם יש בעיה עם התמונה התוכנית צריכה להסתיים.

לדוגמה, בהינתן שה main נמצא בקובץ Shell.java :

java shell cat.jpeg

שימו לב, מומלץ לבדוק אחרי כל הוספה של פונקציונליות כי הממשק אכן עובד.

2.1.1. שימוש במנגנון החריגות:

כאשר הקוד נתקל בשגיאה ויש צורך להדפיס הודעה למשתמש, יש להשתמש במנגנון החריגות של java (מנגנון exceptions). זרקו חריגה מתאימה מהמקום בקוד שבו התגלתה השגיאה, ותפסו את כל החריגות בפונקציה run שם יש להדפיס את ההודעה המתאימה לפי ההוראות בהמשך. השתמשו בחריגות בהתאם לעקרונות שנלמדו בהרצאה ובתירגול, וצרו מחלקות חריגה חדשות לפי הצורך. אתם תתבקשו להסביר ב-README שלכם על הדרך בה הקוד שלכם מתמודד עם טיפול בחריגות, פרטו על מטרת המחלקות שהוספתם.

2.2. קבלת input מהמשתמש ויציאה

הוסיפו פונקציונליות למתודה run כך שכאשר יקראו למתודה, תודפס למסך המחרוזת

">>> "

והמשתמש יוכל לכתוב לתוכה.

<u>שימו לב!</u> השתמשו רק במחלקה KeyboardInput המסופקת בקוד לקבלת קלט מהמשתמש (כמו בתרגיל 1).

בשלב זה, כאשר המשתמש ילחץ enter, נאפשר את שתי האופציות הבאות:

1. לכל מחרוזת השונה מהמחרוזת exit נדפיס שוב את ">>> " וחוזר חלילה.
2. כאשר המשתמש יקליד את המחרוזת exit נסיים את התוכנית.

3. במקרה שבו המשתמש יוסיף קלט מיותר לאחר פקודה חוקית, ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

2.3. צפייה במאגר התווים

הוסיפו למתודה `run` אפשרות שכאשר המשתמש יזין את המחרוזת `chars` נדפיס ל-console את כל התווים שאנחנו יכולים להשתמש בהם בבניית התמונה. ההדפסה תהיה לפי סדר ascii, כל התווים בשורה אחת עם רווח לאחר כל תו. אם המאגר ריק, לא יודפס כלום.

אנא אתחלו את התוכנית כך שבאופן דפולטיבי מאגר התווים יכיל את המספרים 0-9.

במקרה שבו המשתמש יוסיף קלט לאחר הפקודה (עם רווח ביניהם), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

2.4. הוספה למאגר התווים

הוסיפו למתודה `run` את האפשרות להוסיף תווים באמצעות המחרוזת `add`. התווים שאפשר להוסיף הם תווים בעלי ערך ascii בין 32 עד ל-126, כולל.

הפקודה תאפשר להכניס תווים באמצעות שלושת האפשרויות הבאות:

2.4.1. הפקודה `add` ולאחריה תו בודד (מחרוזת באורך 1) – נכניס למאגר שלנו את התו. לדוגמא הפקודה:

```
add a
```

(עלינו להכניס למאגר את התו `a`)

2.4.2. הפקודה `add` ולאחריה המילה `all` .
עבור המילה `all` נכניס למאגר שלנו את כל התווים האפשריים ליצירת התמונה - מהתו ' ' (רווח) עד התו ~. כלומר, מתו הASCII שמספרו 32 עד ל126, כולל.
לדוגמא הפקודה:

```
add all
```

2.4.3. הפקודה `add` ולאחריה אחת המילה `space`. עבור המילה `space` נכניס למאגר שלנו את התו רווח ' '.
לדוגמא הפקודה:

```
add space
```

2.4.4. הפקודה `add` ולאחריה טווח תווים (כל סוג טווח במסגרת תווי ascii, אותיות גדולות, קטנות, מספרים וכדו') מהצורה `a-d` – נכניס למאגר שלנו את כל התווים בטווח `a-d`.
למשל הפקודה:

```
add m-p
```

(עלינו להכניס למאגר את התווים m,n,o,p)

שימו לב שעליכם להתמודד גם עם קלט הפוך, למשל עבור הפקודה:

```
add p-m
```

תצטרכו להכניס למאגר את התווים m,n,o,p.

2.4.5. במקרה שבו המשתמש יוסיף קלט מיותר לאחר פקודה חוקית (אחת מהאפשרויות לעיל), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

2.4.6. הפקודה add ולאחריה כל דבר אחר שלא מפורט לעיל, יחשב כקלט לא תקין ומצופה מכם להדפיס למסך הודעת שגיאה (כולל תווים שלא נמצאים בטווח ascii המוגדר):

```
Did not add due to incorrect format.
```

שימו לב! במידה והמשתמש מבקש להוסיף תו שכבר קיים במאגר, אין צורך להדפיס הודעת שגיאה ויש פשוט להתעלם מהבקשה.

2.5. מחיקה ממאגר התווים

הוסיפו לשיטה run את האופציה להסיר תווים מהמאגר באמצעות המחרוזת **remove** אשר כמו add גם תקבל את כל אחת מהאפשרויות לעיל אבל תסיר מהמאגר את האותיות הנבחרות. גם כאן, במקרה של קלט לא תקין לאחר הפקודה remove, הדפיסו הודעת שגיאה.

```
Did not remove due to incorrect format.
```

ובמקרה של הסרת תו שאינו במאגר – אין צורך להדפיס הודעת שגיאה ויש פשוט להתעלם מהבקשה.

2.6. שליטה ברזולוציה של התמונה שלנו

הוסיפו לשיטה run את האופציה להגדיל או להקטין את הרזולוציה של התמונה באמצעות המחרוזת **res**. פקודה זו יכולה להופיע בלי פרמטר או לקבל אחת משתי מחרוזות up, down:

2.6.1. הפקודה res ללא פרמטר לא תשנה את הרזולוציה ותודפס ההודעה:

```
Resolution set to <current resolution>.
```

עם הרזולוציה הנוכחית.

2.6.2. עבור הפקודה res up נכפיל את מספר התווים בשורה ב-2.

2.6.3. עבור הפקודה res down נחלק את מספר התווים בשורה ב-2.

2.6.4. הדפיסו לאחר השינוי את מספר התווים העדכני למסך באופן הבא:

```
Resolution set to <current resolution>.
```

2.6.5. אתחלו את הרזולוציה הדיפולטיבית, כלומר את מספר התווים ההתחלתיים בשורה, להיות 2. (ניתן להניח שכל התמונות שיתקבלו כקלט לתוכנית יתמכו ברזולוציה הדיפולטיבית)

2.6.6. לכל תמונה, בהתאם לנתוני הרוחב והגובה שלה צריכים להיות ערכים שהם המינימום והמקסימום של מספר תתי-התמונות הריבועיות שאפשר להכניס בשורה, שזה בעצם הרזולוציה המדוברת.
המקסימום הוא מספר הפיקסלים ברוחב התמונה. המינימום יהיה תלוי ביחס הרוחב והגובה של התמונה. ניתן לחשב אותו לפי הנוסחה הבאה:

```
minCharsInRow = max(1, imgWidth/imgHeight)
```

אם בעקבות הפקודה עברתם את הערך המקסימלי/מינימלי, אין לשנות את הרזולוציה, ויש להדפיס את הודעת השגיאה הבאה:

```
Did not change resolution due to exceeding boundaries.
```

בהתאם.

2.6.7. במקרה שבו המשתמש יוסיף קלט מיותר לאחר פקודה חוקית (אחת מהאפשרויות לעיל), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

2.6.8. בכל מקרה של קלט לא תקין לאחר המחרוזת res יש להדפיס הודעת שגיאה:

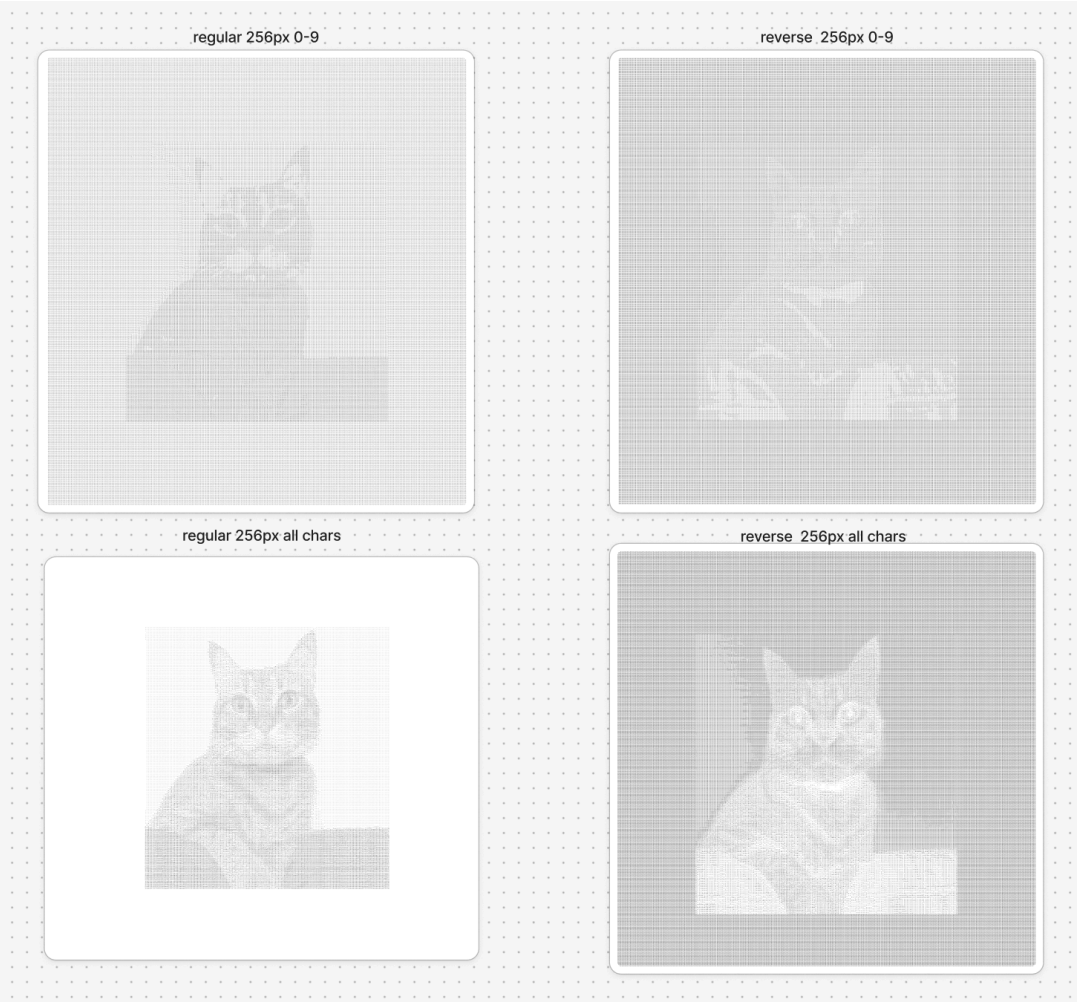
```
Did not change resolution due to incorrect format.
```

2.7. שינוי אופן ההתאמה – מצב Reverse

הוסיפו לשיטה חuז אפשרות להפעיל מצב **reverse**, שבו התאמת התו לתת־התמונה תתבצע באופן הבא:

1. חשבו את בהירות תת התמונה כרגיל.
2. במקום להתאים תו לבהירות המחושבת, חשבו את **הבהירות המשלימה** שלה
3. אם הערך המשלים נמוך מהבהירות המינימלית הקיימת אצלכם אז השתמשו במינימום. אם הוא גבוה מהבהירות המקסימלית השתמשו במקסימום.
4. מצאו את התו שהבהירות המשויכת לו היא **הכי קרובה** לערך המשלֵם.

2.7.1 במקרה שבו המשתמש יוסיף קלט לאחר הפקודה (עם רווח ביניהם), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.



2.8 שינוי שיטת ה-output

הוסיפו לשיטה `run` את הפקודה **`output`** שמשנה את יעד הפלט שלנו. פקודה זו יכולה לקבל אחת משתי מחרוזות console, html. באופן דיפולטיבי, ה-`output` שלנו יהיה הדפסה למסך (console).

2.8.1 עבור הפקודה `output html` יש לדאוג לכך שהפלט של האלגוריתם בריצות הבאות ייכתב לקובץ בשם "out.html" עם הגופן "`Courier New`".

2.8.2 עבור הפקודה `output console` יש לדאוג לכך שהפלט של האלגוריתם בריצות הבאות יודפס למסך.

2.8.3 במקרה שבו המשתמש יוסיף קלט מיותר לאחר פקודה חוקית (אחת מהאפשרויות לעיל), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

2.8.4 הפקודה `output` ללא פרמטר נוסף מהווה קלט לא תקין. בכל מקרה של קלט לא תקין לאחר המחרוזת output יש להדפיס הודעת שגיאה:

```
Did not change output method due to incorrect format.
```

2.9 הרצת האלגוריתם

הוסיפו לשיטה run את הפקודה **asciiArt** אשר מריצה את אלגוריתם ה-ASCII art על פי הפרמטרים המוגדרים כאשר קוראים לה (תמונה, רזולוציה, מאגר תווים). בסיום הריצה הפלט יודפס ליעד שנבחר console או html.

- במקרה של קריאה ל-asciiArt עם מאגר תווים בו פחות משני תווים (תו בודד או מאגר ריק), הדפיסו הודעת שגיאה:

```
Did not execute. Charset is too small.
```

- במקרה שבו המשתמש יוסיף קלט לאחר הפקודה (עם רווח ביניהם), ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל.

* התמודדות עם פקודה לא תקינה

שנו את המתודה `run()` כך שכאשר היא תקבל פקודה לא חוקית (כלומר לא נמצאת ברשימת 8 הפקודות) תודפס הודעת שגיאה:

```
Did not execute due to incorrect command.
```

למשל עבור קלט `adder` הדפיסו את הודעת השגיאה.

שימו ❤️: השתמשו ב**ln**system.out.print על מנת להדפיס את הודעות השגיאה, ההודעה תודפס בשורה אחת בדיוק עם ירידת שורה אחת בלבד בסיום ההדפסה. אין צורך להוסיף \n.

3.הוראות הגשה

3.1. קבצים להגשה

הגישו את התרגיל כקובץ jar/tar/zip בשם **ex3** (והסיומת בהתאם). הקובץ יכיל את הקבצים הבאים (ניתן להוסיף גם חבילות נוספות, או חבילות מקוננות בהתאם לעיצוב שלכם):

1. החבילה **ascii_art** הכוללת את הקבצים
   - **`AsciiArtAlgorithm.java`**
   - **`Shell.java`**
   - KeyboadInput.java - אין להגיש קובץ זה
   - קבצים נוספים בהתאם לעיצוב שבחרתם.
2. החבילה **image** הכוללת את הקובץ
   - **`Image.java`**
   - קבצים נוספים בהתאם לעיצוב שבחרתם.
3. החבילה **image_char_matching** הכוללת את הקבצים:
   - **`SubImgCharMatcher.java`**
   - **`charConverter.java`**
   - קבצים נוספים בהתאם לעיצוב שבחרתם.
4. קובץ בשם **README** (ללא סיומת) לפי ההוראות בסעיף 3.2.
5. קובץ **UML.pdf** המכיל דיאגרמת UML המתארת את הקוד שלכם. יש לציין בדיאגרמה את כל המחלקות שהתבקשתם להגיש. אין צורך לציין מחלקות של חריגות (exceptions).
   יש להגיש את הקובץ בפורמט PDF ולוודא שהדיאגרמה נכנסת במלואה בקובץ submission.pdf הנוצר בהגשה!

שימו ❤️: אין להגיש את החבילה ascii_output.

3.2. קובץ README

צרפו להגשה שלכם קובץ README הכולל את הפרטים הבאים:

- בשורה הראשונה בקובץ יופיע שם המשתמש CSE שלכם.
  אם אתם מגישים בזוג, יש להפריד בין שמות המשתמש עם פסיק (שניהם באותה שורה).
- בשורה השניה מספר תעודת הזהות.
  אם אתם מגישים בזוג, יש להפריד בין מספרי הזהות עם פסיק (שניהם באותה שורה).
- השורה השלישית ריקה.

- **<u>החל מהשורה הרביעית ענו בקובץ על השאלות הבאות לפי הסדר</u>**:

1. בהתאם לדיאגרמת UML של הקוד שיצרתם, הסבירו בקצרה מה תפקידה של כל מחלקה בקוד שלכם ועל הקשרים בין המחלקות.
2. הסבירו בקצרה עבור כל מבנה נתונים של java בו השתמשתם בקוד, היכן השתמשתם בו ולמה בחרתם דווקא במבנה נתונים זה עבור המשימה. התייחסו בהסבר לשיקולי סיבוכיות זמן ריצה וזיכרון.
3. הסבירו איך השתמשתם במנגנון החריגות של java על מנת לטפל בשגיאות בקלט המשתמש והדפסת הודעות שגיאה.
4. אם הכנסתם שינויים ב-API של המחלקה `SubImgCharMatcher` הסבירו למה השינויים היו נצרכים בקוד.
5. אם הכנסתם שינויים בקוד של המחלקות שסופקו לכם הסבירו את השינויים ולמה הם היו נצרכים. **אין להכניס שינויים בקוד של המחלקות בחבילה ascii_output שכן לא מגישים חבילה זו!**

3.3. דוגמה לקובץ פלט

סיפקנו לכם בתיקיית חומר העזר קבצי קלט ופלט לדוגמת החתול עבור מקרים 1 ו-2 מתוך ההקדמה לתרגיל. הקובץ הראשון בשם out1 התקבל עבור רזולוציה 128 עם התווים 0-9 ועיגול דיפולטיבי של ערך מוחלט. והקובץ השני בשם out2 התקבל עבור רזולוציה 256 עם כל התווים האפשריים ועיגול דיפולטיבי של ערך מוחלט. תוכלו להשתמש בקבצים אלו לצורך בדיקה עצמית.

בהצלחה!

<u>נספח שאלות נפוצות והמלצות</u>

כתוב שהרזולוציה מגדירה את מספר תתי התמונות בכל שורה בתמונה, אך בסעיפים אלה כתוב שאנו משנים את מספר התווים בשורה. אשמח להבהרה.

<u>הכוונה לשנות את מספר התתי תמונות</u>

בסעיף 1.4.3 כתבתם שעבור כל הרצה של האלגוריתם יש ליצור מופע חדש של המחלקה.

כלומר הוא יקבל "מחדש" את השם התמונה ואת הרזולוציה וכו' ואין לו שום מושג מה היו הפרמטרים של המופע הקודם אז איך בדיוק הוא אמור "לזכור" את הנתונים שנשמרו בריצה הקודמת? הוא מופע חדש ועצמאי לחלוטין

אפשר ליצור מופע חדש של התחלקה AsciiArtAlgorithm ועדיין לזכור חישובי בהירות שנעשו קודם. יש לחשוב על עיצוב מתאים

לגבי סעיף 1.5.1-1.5.3, רציתי לשאול האם מותר להשתמש בפונקציה Arrays.deepHashCode() והאם היא נחשבת בטוחה ועובדת כדי לעשות Hash למערכים שאנחנו לא נשנה את הערכים בהם אף פעם.

בעיקרון אפשר אבל באחריותך לוודא אם זה מתאים\עובד וכו

מכיוון שלא נדרש מאיתנו API ספציפי לפי סעיף זה, האם ניתן לשנות את image.java שסופק כרצוננו?

מותר לעשות שינויים אבל מבחינה עיצובית עדיף להשאיר אותה בלי שינויים מכוון ש לimage.java יש תפקיד ברור ומוגדר היטב.

סעיף 2.6.6 האם הכוונה היא לרוחב התמונה המקורית או רוחב התמונה לאחר padding?

למשל עבור תמונה ברוחב 640 לאחר padding המונה תהיה ברוחב 1024 והרזולוציה המקסימלית שלה תהיה 1024 בהתאם לכך.

מכוון שהחלוקה נעשת אחרי הפדינג הרזולוציה המקסימלית היא בהתאם לתומונה אחרי הפדינג

סעיף 1.4.1.2 האם יש הכרח שמבנה הנתונים שלנו יחזיק בכל זמן נתון את ערך הבהירות לפי newCharBrightness? או שניתן להחזיק את הבהירות כפי שחישבנו בתת סעיף 2, ורק בהחזרה הרלבנטית לחשב לפי newCharBrightness?

היי ניתן להחזיק את הבהירות כפי שחישבתם בתת סעיף 2, ורק בהחזרה הרלבנטית לחשב לפי newCharBrightness

סעיף 1.4.1.2 האם יש צורך לנרמל את התווים בכל פעם שמוסיפים תו (למעשה באמצעות addChar)?

(נראה שבתהליך הבניה עצמו לא נכון לעשות זאת, אבל האם לאחר מכן כך צריך?)

כמו כן האם יש צורך לנרמל מחדש במידה והסרנו תו באמצעות removeChar()?

<u>לתווים יש בהירות מנורמלת שצריך להתייחס אליה בהרצת האלגוריתם. מתי הנירמול מתבצע זה בחירה שלך</u>

סעיף 1.4.3 אשמח לדעת מה מדיניות העיגול הרצויה בערך מוחלט במקרה שבו המרחק של הערך העליון ושל הערך התחתון מהערך המבוקש זהה. האם לקחת את התחתון או את העליון?

<u>היי אפשר להניח שלוקחים את התחתון</u>

סעיף 2.3 בהוראות רשום "במקרה שבו המשתמש יוסיף קלט לאחר הפקודה עם רווח ביניהם, ניתן להתעלם משאר הקלט ולבצע
את הפקודה כרגיל. "

האם זה חובה להתעלם מכך ? או שאם אני פותרת את הנושא כך שיזרוק הודעות שגיאה גם אז זה עדיף?

<u>חובה להתעלם</u>

סעיף 2.4.6 בהוראות התרגיל רשום: "הפקודה add ולאחריה כל דבר אחר שלא מפורט לעיל, יחשב כקלט לא תקין ומצופה מכם
להדפיס למסך הודעת שגיאה כולל תווים שלא נמצאים בטווח ascii המוגדר"

האם טווח הASCII המוגדר הוא מ32 ועד 126 או משהו אחר?

<u>טווח הASCII המוגדר הוא מ32 ועד 126 ולכן זה הקלטים התקינים</u>

<u>סעיף 2.4.5</u>

שלום, בסעיף 2.4.5 כתוב: "במקרה שבו המתשמש יוסיף קלט מיותר לאחר פקודה חוקית, ניתן להתעלם משאר הקלט ולבצע את הפקודה כרגיל".

האם הכוונה היא שהפקודה:

add a-z1

תיחשב תקינה ועל כן להוסיף את התווים a-z? או שזו פקודה לא תקינה, והכוונה היא שאם אחרי פקודה חוקית מופיע קלט מיותר המופרד ברווח אפשר להתעלם, כמו ב-

add a-z 1

וכאן אין ספק שצריך להוסיף את התווים a-z.

"add a-z1" צריך לגרום לשגיאה בניגוד ל"add a-z 1" שצריך להתבצע

סעיף 2.6

נשמח לדעת מה זה אומר קלט לא תקין לאחר המחרוזת RES.

כי בסעיף 2.6.7 נאמר כי קלט מיותר לאחר פקודה חוקית הוא תקין, והפקודות החוקיות הם כלום אפ או דאון, ולכן לא משנה איזה קלט נכניס זה ייכנס לאחד המקרים האלה בתור קלט מיותר. אז יוצא שאין קלט לא תקין.

במקרה הספציפי של res הינה ההתנהגויות המצופות

res up תקין

res תקין

res up t תקין

res t לא תקין

res t לא תקין

סעיף 2.4

האם ניתן להניח שלא יתקבל בקלט add ללא מידע נוסף או שמא צריך לטפל גם במקרה זה באמצעות החזרת אותה שגיאה?

add ללא ארגומנט צריך להחזיר שגיאה

סעיף 2.9 האם בסעיף זה אני אמור לעצור את ריצת התוכנית עבור פקודה לא חוקית או להמשיך ולקבל פקודה חדשה?

<u>היי במקרה של שגיאה לא עוצרים את התוכנית. מדפיסים את השגיאה וממתינים לפקודה חדשה</u>

סעיף 2.5

1. במידה והמשתמש מבקש להסיר מהמאגר תו שלא נמצא בטבלת ASCII (ולכן בהכרח לא נמצא במאגר), האם מדובר בקלט לא תקין או שיש להתעלם מהבקשה?

2. במידה ויש להתעלם מהבקשה, מה לעשות במקרה שהמשתמש רוצה להסיר טווח תווים כאשר אחד או יותר התווים בטווח לא בטבלת ASCII, מה ההתנהגות המצופה במקרה כזה?

<u>היי צריך להתעלם מהבקשה</u>

איך אנחנו מטפלים במצב ששתי אותיות הן בעלות אותו ערך בהירות?

<u>אפשר להניח שאחרי כל הנירמולים ב.1.4.1.2 אין מצב ששתי אותיות חולקות את אותה בהירות</u>

סעיף 2.4.4

האם ייתכן מצב בו נקבל קלט שכולל בתוכו גם תווים חוקיים וגם לא חוקיים לדוגמא F5-C?

ואם כן, האם זה נחשב תקין ועלינו לצרף רק את התווים החוקיים בטווח? או האם עלינו להתייחס לכך כקלט לא תקין בכלל?

<u>אפשר לא להתייחס לתווים לא חוקים</u>

סעיף 2.1

היי, האם ניתן להניח תקינות ארגומנטים שמתקבלים ב-main? כלומר ארגומנט אחד שהוא נתיב לתמונה (נתיב תקין / לא תקין)

במידה שלא, מה מצופה מאיתנו להדפיס במקרה של נתיב לא תקין (בין אם לא קיבלנו נתיב בכלל או שהוא לא תקין)?

בסעיף 2.1 כתוב "אם יש בעיה עם התמונה התוכנית צריכה להסתיים" - כלומר ללא שום הדפסה?

<u>אפשר להניח קלט תקין</u>

<u>אם יש בעיה עם התמונה התוכנית צריכה להסתיים. אתם בוחרים איך זה יקרה בדיוק ולא משנה מה ההודעה שתודפס.</u>

סעיף 2.8.1

אנחנו אמורים לשמור את קובץ הhtml בתיקייה שבה התכנית רצה?

<u>כן. לא צריך להוסיף נתיב לשם הקובץ.</u>

סעיף 2.1 האם אנחנו אמורים לאכוף שהתכנית תקבל בדיוק ארגומנט אחד? מה קורה עם אנחנו מקבלים שתיים ומעלה?

<u>לא צריך לטפל במקרה כזה במסגרת התרגיל</u>

סעיף 2.3 האם צריך להדפיס ירידת שורה לאחר הדפסת כל המאגר (עם הרווחים ביניהם) ולפני בקשת הפקודה הבאה?

<u>כן</u>

<u>סעיף 1.4.3</u>

איזה ערך בהירות אמורים להחזיר במקרה ששיטת העיגול שנבחרה היא "למעלה", וערך הבהירות שקיבלנו גבוה מערך הבהירות הגבוה ביותר שקיים כרגע במבנה הנתונים שלנו?

באופן דומה מה אמורים להחזיר במקרה ששיטת העיגול שנבחרה היא "למטה", וערך הבהירות שקיבלנו נמוך מערך הבהירות הנמוך ביותר שקיים כרגע במבנה הנתונים שלנו?

<u>את ערך הבהירות הגבוה/הנמוך ביותר בהתאם.</u>

שאלה בהמשך - סעיף 1.4.2.2

אשמח לעזרה בנוגע לסעיף הנ"ל - רשום שתת כל תמונה צריכה להיות ריבוע של פיקסלים, אבל התמונה שנקבל כקלט לא בהכרח חייבת להיות ריבוע, אז לא הצלחתי להבין איך צריך לבצע את החלוקה.

<u>אפשר לחלק לריבועים גם אם התמונה לא ריבועית עצמה. ראה ריפוד ב1.4.2.1</u>

מה קורה במקרה בו לצורך העניין התמונה היא ברוחב 32 וגובה 8 והרזולוציה היא 2?

במקרה זה להבנתי ישנן שתי תמונות בשורה כלומר 16 פיקסלים אך אין אפשרות לתת תמונה ריבועית שכן אין 16 פיקסלים באורך התמונה המקורית.

האם הנתון כי בסעיף זה ניתן להניח כי הרזולוציה הנתונה תקינה מתייחס גם למקרים אלו או שמתייחס רק לתקינות הרזולוציה (כלומר מספר חיובי, חזקה של 2 וכו?)

<u>כן אפשר להניח שהרזולוציה היא תקינה</u>

"מה קורה במקרה בו לצורך העניין התמונה היא ברוחב 32 וגובה 8 והרזולוציה היא 2"

לא יוכל לקרות? או האם לא הבנתי נכון?

<u>נכון. לא נבדוק אתכם על מקרים כאלו.</u>

האם במקרה של קלט נוסף אחרי פקודת exit או output (מופרד ברווח, למשל - exit add),

גם כן יש להתעלם מהקלט הנוסף ולצאת?
או שבמקרה הזו מדובר על קלט לא-תקין?

(שואל בגלל שבפקודות add, remove, chars, res, round, asciiArt כן נכתב במפורש להתעלם מהקלט, אבל בפקודות exit, output לא נכתב כלום)

<u>אפשר להתעלם מהקלט הנוסף גם במקרים אלו</u>

סעיף 1.4.4.1 - ערך ההחזרה מנורמל?

אשמח בבקשה להבין האם המתודה מתייחסת לערכי הבהירות של התווים אחרי הנרמול הלינארי או לפני?

<u>ההתאמה היא לערך אחרי הנרמול.</u>

מה רמת הפירוט שצריכה להיות בדיאגרמת UML?

אני רואה בתרגול כשלמדנו את זה הסבר כללי ולכן אני שואל.

1. כל המחלקות והמשתנים הפרטיים צריכים להיות בדיאגרמה? אז אם יש לי המון קבועים הם גם צריכים להופיע?

2. האם במתודה, צריכים להופיע הארגומנטים שלה גם בטיפוס וגם בשם או רק בטיפוס?

לדוגמה:

(operation(int x) OR operation(int

3. למשתנים סטטיים יש נוטציה מיוחדת? לא מצאתי לזה אזכור במצגת התרגול

4. והאם צריך להוסיף גם את המחלקות שנתתם לנו לדיאגרמה כמו המחלקות בasciioutput?

<u>1. מחלקות כן משתנים לא.</u>

2. לא.

3. לא צריך לציין משתנים.

4. לא.

היי, התבקשנו להוסיף מחלקת חריגות לפי הצורך.

השאלה היא האם חייב לכתוב בעצמנו את כל החריגות כמחלקות או שאפשר בחלק מהמקרים להסתפק בחריגות המוכרות כמו IOExceptions במקומות המתאימים?

אפשר.

1. האם צריך להוסיף קבצים שהם לא הגשה בUML? לדוגמא את ascii_output שמסופק לנו, או KeyboardInput ב-ascii_art.

2. לא כתוב האם להגיש את הקובץ המסופק לנו charConverter (בחבילה image_char_matching:נ).

האם להגיש אותו או לא?

1. לא

2. עריכה - כן.

היי, רציתי לשאול איך האם אמורים להתייחס למקרה הבא:

add CC

כאשר אין רווח בין הC הראשונה והשנייה.

אמורים לקבל רק את הC הראשונה? או צריך להדפיס שגיאה?

זה קלט לא תקין.

אפשר להתעלם מהמשך הקלט רק אם יש רווח בין החלק התקין לשאר הקלט.

סעיף 2.9 האם למשל פקודה -

"adddddd" נחשבת לפקודה לא תקינה ואז נדפיס

"Did not execute due to incorrect command."

או שזה נחשב משום מה בתור פקודת add לא חוקית ועבורה נתבקש להדפיס ע"פ 2.4.6

"Did not add due to incorrect format."

להבנתי האפשרות הראשונה היא הנכונה אז ברצוני לוודא.

נכון

שלום, אני מקבל את השגיאה הזו בהגשת התרגיל על מחסור בדוקומנטציה על ENUM

יש לי תיעוד למחלקה עצמה, אז אשמח לעזרה להבין מה אני מפספס.

יש לתעד כל אחד מהאובייקטים ב-enum גם כן.